FACULTAD DE INGENIERÍA

Carrera de Ingeniería de Sistemas Computacionales

Sistema de pedidos e inventario para la Quesería Los Alpes

Informe técnico de proyecto final

Fundamentos de Programación CIIN1205P

Grupo 1

Docente: Pereda Cabanillas, Edwuard Jhonatan

Cajamarca, Perú
2026-2

Integrantes y porcentajes de participación

_____
_____
_____
_____
_____

1 Introducción y alcance del proyecto

El proyecto propone un programa de consola para registrar los pedidos de la Quesería Los Alpes, comprobar la disponibilidad de sus productos y conservar las ventas en un archivo. La prioridad es que una venta aceptada tenga un importe correcto y que el inventario no se reduzca cuando el pedido se rechaza. A estas funciones se suman la búsqueda por código, el ordenamiento de pedidos y un resumen de la recaudación diaria.

El caso comercial se desarrolla con autorización del docente, según lo indicado por el grupo. Se conserva la exigencia técnica del desafío CIIN1205P: analizar un proceso, comparar soluciones, diseñar algoritmos, implementar funciones y arreglos, trabajar con cadenas y archivos, y comprobar los resultados. Los expedientes del caso municipal se sustituyen por pedidos identificados mediante un código único. Este prototipo comercial no se presenta como una mesa de partes ni como una implementación de la normativa de gobierno digital.

Objetivo general

Desarrollar y verificar un prototipo en Python que permita registrar, validar, consultar, ordenar y almacenar pedidos, manteniendo la relación entre las ventas y las existencias de cada producto.

Objetivos específicos

- Describir el flujo de atención y distinguir las actividades que realiza el programa de las tareas físicas del personal.
- Validar los datos del cliente, el canal de atención, los productos y las cantidades antes de modificar el inventario.
- Calcular los importes con precisión decimal, asignar códigos únicos y conservar los datos entre sesiones.
- Evaluar el comportamiento con casos normales, límites de stock y fallas de entrada o almacenamiento.

Alcance y límites

La versión incluye queso fresco, queso mantecoso, queso suizo y mantequilla. Permite varios productos por pedido y funciona en una computadora, con una sola instancia abierta. No incorpora reposición, cambios de precio, anulaciones, crédito, impuestos, facturación electrónica ni varios cajeros simultáneos. La constancia emitida sirve para control interno; no reemplaza un comprobante tributario.

El informe base describe el negocio y aporta ejemplos de ventas. Esos ejemplos se utilizan como datos de demostración. Las fotografías, las notas de entrevista, las mediciones de campo y los datos de los integrantes se completarán en los anexos correspondientes.

2 Proceso de atención y análisis del problema

Descripción del caso

El informe base sitúa a la Quesería Los Alpes en el jirón Junín 965, Cajamarca, y describe una atención presencial y telefónica. El personal recibe el pedido, consulta las existencias, anota la información en un cuaderno, prepara el producto, calcula el importe y registra la venta. La elección del caso se justifica porque estas operaciones permiten aplicar validaciones, cálculos, búsquedas y actualización de datos a un proceso comercial concreto.

El mismo informe menciona consultas de inventario de tres a siete minutos y esperas de hasta 35 minutos en horas de mayor demanda. Se conservan como estimaciones del antecedente, no como mediciones verificadas ni como base para afirmar un porcentaje de mejora. La ficha de observación del anexo permite contrastarlas posteriormente.

Secuencia del proceso

Paso	Actividad del personal	Apoyo del programa
1	Recibir el pedido y conocer el canal.	Capturar cliente, canal, producto y cantidad.
2	Confirmar existencias físicas.	Consultar el saldo registrado y validar la cantidad.
3	Confirmar el pedido.	Calcular el total y guardar el registro.
4	Cortar, pesar y empaquetar.	No ejecuta tareas físicas ni verifica la balanza.
5	Cobrar y entregar.	Mostrar una constancia interna.
6	Revisar el cierre.	Consultar ventas del día e inventario restante.

Causas del problema según el enfoque de las seis M

Dimensión	Causa o riesgo identificado
Personal	La atención depende de que cada vendedor anote todos los datos.
Equipos	El cuaderno no realiza búsquedas automáticas ni cálculos.
Entorno	La concurrencia de clientes dificulta actualizar el registro a tiempo.
Datos	Los apuntes pueden quedar incompletos o resultar difíciles de leer.
Métodos	La consulta, el cálculo y la actualización se hacen por separado.
Gestión	No existe una conciliación automática entre pedidos y stock.

El efecto común es la pérdida de confiabilidad del registro. El software puede prevenir errores de digitación y cálculo; la concordancia con el stock físico sigue requiriendo controles del personal.

3 Requerimientos y modelo de datos

Entrada	Proceso	Salida
Cliente y canal	Normalizar espacios y validar contenido.	Datos aceptados o mensaje de corrección.
Producto y cantidad en kg	Agrupar productos repetidos y comprobar stock.	Pedido válido o rechazo sin cambios.
Precio y stock del catálogo	Calcular subtotales y descontar cantidades.	Importe y saldo actualizado.
Código del pedido	Recorrer la lista y comparar códigos.	Detalle del pedido o aviso de no encontrado.
Lista de pedidos y fecha	Ordenar por código y filtrar ventas del día.	Listado ordenado y resumen diario.
Archivo JSON	Leer, validar y guardar registros.	Datos disponibles al reiniciar.

Datos y reglas de validación

Dato	Tipo y regla
Cliente	Cadena de 2 a 80 caracteres; letras, espacios, apóstrofo y guion. Se reducen espacios repetidos.
Canal	Cadena; presencial o telefónico, sin distinguir mayúsculas.
Producto	Identificador entero del 1 al 4. El nombre y precio se toman del catálogo.
Cantidad	Decimal positivo; hasta tres decimales y seis cifras enteras. Acepta coma o punto.
Stock y precio	Stock no negativo y precio positivo. Son datos del catálogo de demostración.
Código	Cadena PED-000001 a PED-999999; consecutivo generado por el sistema.
Fecha y total	Fecha local con zona horaria; total monetario con dos decimales.

Cada pedido contiene una lista de líneas con producto, kilogramos, precio unitario y subtotal. Los importes se guardan como cadenas decimales en JSON para recuperar su valor sin convertirlos a números binarios de punto flotante. El estado reúne dos listas: productos y pedidos.

Criterios de aceptación

Un pedido válido se guarda y recibe un código; uno inválido conserva el estado anterior. La búsqueda devuelve el código solicitado, el ordenamiento no pierde registros y un reinicio recupera el último pedido confirmado. Si el archivo está dañado, la aplicación se detiene con un aviso y lo conserva para revisión.

4 Alternativas y selección de la solución

Se compararon dos soluciones algorítmicas implementables por el equipo y la opción de contratar un servicio externo. La comparación utiliza criterios técnicos; no supone que se haya solicitado una cotización.

Criterio	A Lista y búsqueda lineal	B Índice y búsqueda binaria	C Servicio externo
Organización	Lista de pedidos y JSON.	Lista ordenada e índice por código.	Depende de la propuesta del proveedor.
Consulta	Recorre hasta n pedidos.	Hasta $\log_2 n$ pasos en lista ordenada.	Debe verificarse en una demostración.
Ordenamiento	Inserción explícita, fácil de explicar.	Mantener orden tras cada alta.	Depende del sistema contratado.
Recursos	Python y almacenamiento local.	Python y mayor cuidado al mantener índices.	Contratación, configuración y soporte.
Viabilidad académica	Alta para el nivel del curso.	Posible, con mayor complejidad.	Menor evidencia de programación propia.
Costo	Tiempo de desarrollo y mantenimiento.	Más tiempo de diseño y pruebas.	Cotización inicial y posibles pagos periódicos.

Valoración de las alternativas propias

Se asignó una escala de 1 a 5, donde 5 es más favorable. Los pesos y puntajes son una valoración de diseño para este prototipo, no resultados de una encuesta: cumplimiento 35 %, facilidad de explicación 30 %, recursos disponibles 20 % y mantenimiento 15 %.

Alternativa	Cumple	Explicable	Recursos	Mantener	Puntaje
A	5	5	5	4	4,85
B	5	3	4	3	3,90

Se seleccionó la alternativa A. Su estructura hace visible la lógica de búsqueda y ordenamiento y permite concentrar el trabajo en la integridad del pedido. El menor rendimiento del ordenamiento por inserción se acepta para un volumen académico reducido; no se afirma que sea adecuado para un historial comercial de gran tamaño.

Contraste con tercerizar

Una contratación podría incorporar soporte, múltiples usuarios y facturación, pero habría que evaluar el costo total, la propiedad de los datos y la exportación del historial. Sin una cotización no corresponde asignarle un importe. Para comparar económicamente se propone sumar instalación, capacitación, soporte y pagos recurrentes durante el mismo periodo. La necesidad inmediata del curso se cubre mejor con un prototipo propio que pueda explicarse y modificarse.

5 Principios de programación y organización

Relaciones matemáticas e invariantes

Para cada producto, el subtotal se obtiene como cantidad × precio por kilogramo y se redondea a dos decimales. El total del pedido es la suma de sus subtotales. La regla de inventario es stock final = stock inicial − cantidad vendida, con la condición 0 < cantidad ≤ stock inicial. Si un producto aparece varias veces en el pedido, sus cantidades se suman antes de comprobar esa condición.

Ejemplo: 15 kg de queso fresco a S/ 18 por kg generan S/ 270,00. Con 20 kg iniciales quedan 5 kg. Un pedido de 20 kg de mantecoso con solo 15 kg disponibles se rechaza y mantiene el saldo en 15 kg. En una compra de 5 kg de suizo y 5 kg de mantequilla, el total es 5 × 28 + 5 × 25 = S/ 265,00.

Descomposición del sistema

Módulo	Responsabilidad y funciones
main.py	Interacción de consola, menú y presentación de resultados.
negocio.py	numero, inicial, registrar, buscar, ordenar y resumen. Contiene las reglas de la venta.
archivos.py	cargar, validar y guardar. Mantiene el estado en JSON.
test_prototipo.py	Verifica resultados con un estado independiente y archivos temporales.

Las funciones reciben parámetros explícitos. registrar devuelve el pedido confirmado; buscar devuelve un registro o None; ordenar devuelve una nueva lista. guardar escribe el archivo y no devuelve un resultado de negocio. Las variables de cálculo son locales. La ruta del archivo se define en la interfaz y se entrega a la función de persistencia.

Estructuras del curso

La secuencia se observa al capturar, validar, calcular y guardar. Los condicionales rechazan entradas y seleccionan opciones; los bucles mantienen el menú y recorren productos. Las listas representan arreglos dinámicos: se usan arreglos paralelos para construir el catálogo y listas de registros para los pedidos. Cada pedido contiene una lista de detalles, equivalente a una estructura tabular de varias filas. Las cadenas se limpian con strip, split y join; los códigos se normalizan con upper.

Eficiencia y verificación

Para n pedidos, la búsqueda lineal necesita como máximo n comparaciones. El ordenamiento por inserción tiene costo cuadrático $O(n^2)$ en el peor caso y lineal $O(n)$ si la lista ya está ordenada. La copia y escritura del estado aumentan con el historial completo. La comprobación consiste en contrastar las salidas con resultados calculados de antemano y verificar que los rechazos no alteren los datos.

6 Diagrama de flujo del programa

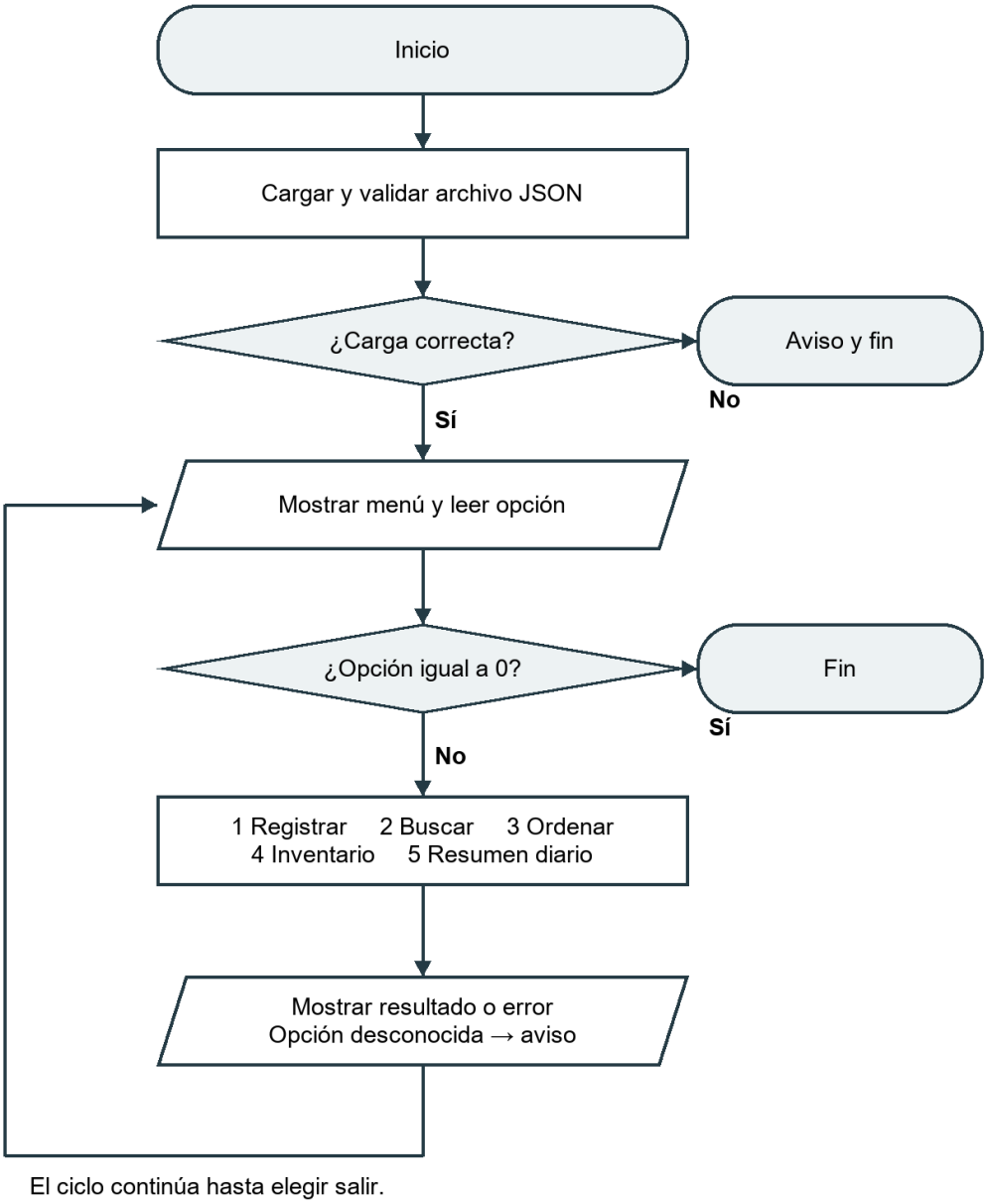


Figura 1. Flujo general del prototipo. El óvalo indica inicio o fin, el paralelogramo entrada o salida, el rectángulo proceso y el rombo decisión.

La carga del archivo se realiza antes de mostrar el menú. Un archivo inexistente permite iniciar el catálogo de demostración; un archivo inválido impide continuar. Cada operación finaliza en el menú y los errores de entrada se informan sin cerrar la sesión.

7 Diagrama de flujo del registro de pedidos

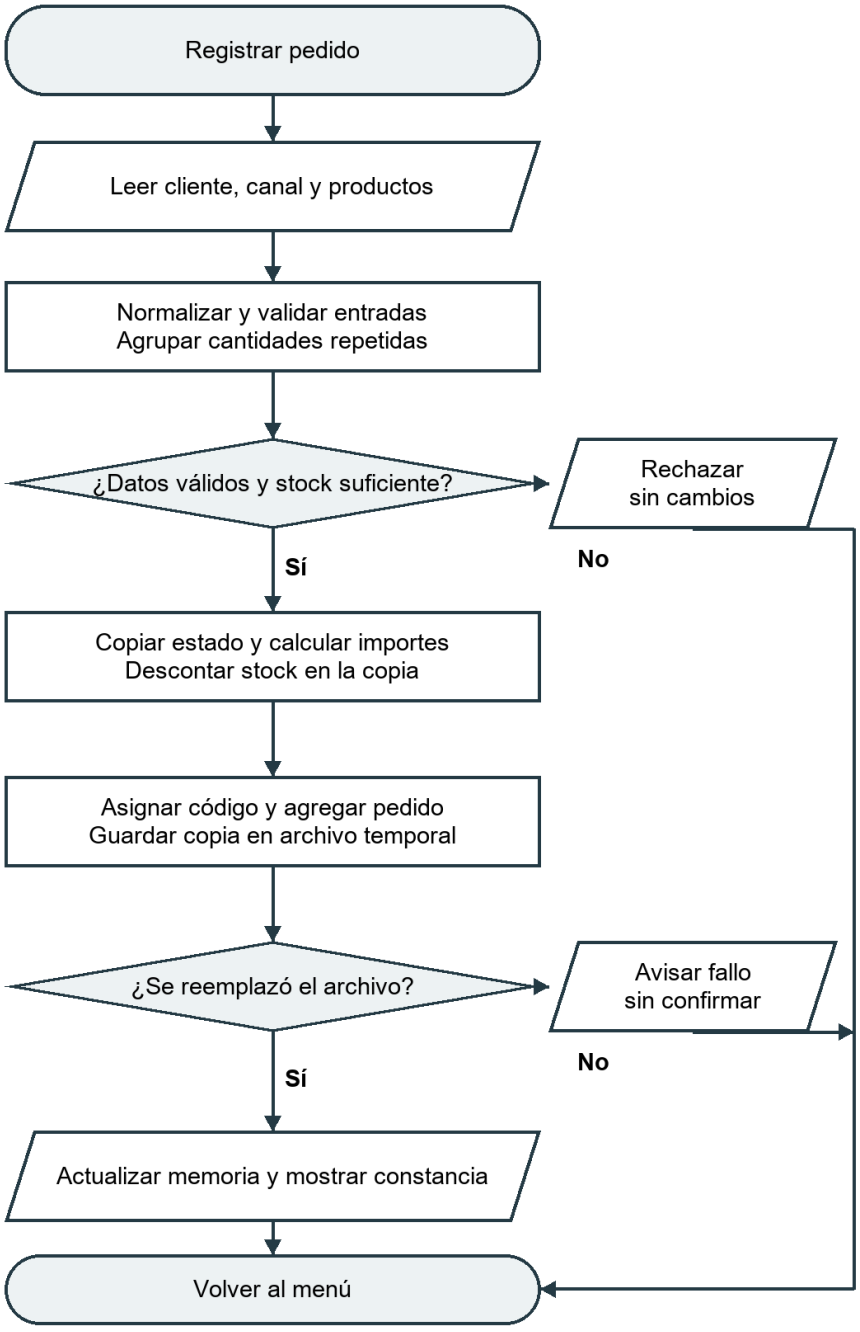


Figura 2. Registro de un pedido. La actualización en memoria ocurre después del guardado correcto.

El pedido se procesa sobre una copia del estado. Si una línea supera las existencias o falla el almacenamiento, la venta completa se rechaza. Esta secuencia evita descuentos parciales cuando un pedido contiene varios productos.

8 Pseudocódigo del sistema

```
INICIO
  INTENTAR cargar y validar el archivo
  SI no existe, crear catálogo inicial y lista vacía
  SI hay error de lectura o validación, informar y TERMINAR
  REPETIR
    Mostrar menú y leer opción
    SEGÚN opción
      1: Leer cliente, canal y líneas; REGISTRAR
      2: Leer código; BUSCAR y mostrar resultado
      3: ORDENAR copia de pedidos y mostrarla
      4: Mostrar catálogo y stock
      5: Filtrar fecha de hoy; contar pedidos y sumar totales
      0: Terminar
      OTRO: Informar opción inválida
    Si ocurre error de entrada o escritura, mostrar aviso
  HASTA opción = 0
FIN

REGISTRAR cliente, canal, líneas
  Normalizar nombre y canal; validar valores
  Rechazar lista de líneas vacía
  Agrupar las cantidades de cada producto
  Copiar el estado actual
  PARA cada producto solicitado
    Validar producto y cantidad positiva
    SI cantidad > stock, rechazar el pedido completo
    subtotal ← redondear(cantidad × precio, 2)
    stock de la copia ← stock – cantidad
    Agregar detalle y acumular subtotal
  FIN PARA
  código ← máximo código existente + 1
  Rechazar si se supera el límite de seis cifras
  Agregar pedido con fecha, cliente, canal, detalle y total
  GUARDAR la copia
  SI guardado exitoso, sustituir estado y devolver pedido

BUSCAR pedidos, código
  Normalizar código quitando espacios y usando mayúsculas
  PARA cada pedido
    SI su código coincide, devolver pedido
  FIN PARA
  Devolver no encontrado
```

9 Pseudocódigo de ordenamiento y persistencia

```
ORDENAR pedidos
  copia ← lista de pedidos
  PARA i desde 1 hasta longitud(copia) − 1
    actual ← copia[i]; j ← i − 1
    MIENTRAS j ≥ 0 Y copia[j].código > actual.código
      copia[j + 1] ← copia[j]
      j ← j − 1
    FIN MIENTRAS
    copia[j + 1] ← actual
  FIN PARA
  Devolver copia

GUARDAR ruta, estado
  Validar estructura, códigos e importes
  Crear archivo temporal en la misma carpeta
  Escribir estado completo como JSON en UTF-8
  Vaciar búfer y sincronizar escritura
  Reemplazar archivo de destino por el temporal
  Eliminar temporal sobrante si ocurre un error

CARGAR ruta
  SI archivo no existe, devolver estado inicial
  Leer JSON y validar estructura y datos
  SI contenido inválido, informar y conservar archivo
  Devolver estado validado
```

Elección del formato de archivo

Se eligió JSON de texto porque conserva la relación entre productos, pedidos y detalles y permite inspeccionar el archivo durante las pruebas [2]. Un CSV simplifica las filas, pero obliga a separar encabezados y detalles o a repetir datos. Un archivo binario puede reducir legibilidad y requiere herramientas específicas para su revisión. Para este prototipo, la facilidad de verificar el contenido pesa más que reducir el tamaño.

El guardado utiliza un temporal en la misma carpeta y os.replace para sustituir el destino [3]. Esto reduce el riesgo de dejar un archivo a medio escribir. No equivale a un sistema de respaldo ni resuelve escrituras simultáneas: se requiere una sola instancia, permisos de escritura y copias de seguridad independientes.

Condiciones que deben mantenerse

Los códigos son únicos, ningún stock es negativo y cada total coincide con la suma de detalles. Un fallo de lectura no crea silenciosamente un inventario nuevo. Al guardar, la memoria solo se actualiza después de confirmar la escritura. El catálogo de demostración no representa las existencias físicas actuales de la quesería.

10 Implementación del registro y control de errores

```
    if not lineas:
        raise ValueError('El pedido está vacío.')
    nuevo = deepcopy(estado)
    cantidades = {}
    for identificador, cantidad in lineas:
        if type(identificador) is not int:
            raise ValueError('Producto inválido.')
        cantidades[identificador] = cantidades.get(identificador, Decimal('0')) +
numero(cantidad)
    detalle, total = [], Decimal('0')
    for identificador, cantidad in cantidades.items():
        producto = next((p for p in nuevo['productos'] if p['id'] == identificador), None)
        if producto is None:
            raise ValueError('Producto inexistente.')
        stock = numero(producto['stock'], cero=True)
        if cantidad > stock:
            raise ValueError('Stock insuficiente para ' + producto['nombre'])
        subtotal = (cantidad * numero(producto['precio'])).quantize(Decimal('.01'),
rounding=ROUND_HALF_UP)
        producto['stock'] = str(stock - cantidad)
        detalle.append({'producto': identificador, 'nombre': producto['nombre'],
                        'kg': str(cantidad), 'precio': producto['precio'], 'subtotal':
str(subtotal)})
        total += subtotal
    siguiente = max((int(p['codigo'][4:]) for p in estado['pedidos']), default=0) + 1
    if siguiente > 999999:
        raise ValueError('Se alcanzó el límite de códigos del prototipo.')
    pedido = {'codigo': f'PED-{siguiente:06d}', 'fecha':
datetime.now().astimezone().isoformat(timespec='seconds'),
              'cliente': cliente, 'canal': canal, 'detalle': detalle, 'total': str(total)}
    nuevo['pedidos'].append(pedido)
    guardar(nuevo)  # Solo se confirma después de guardar correctamente.
    estado.clear()
    estado.update(nuevo)
    return pedido
```

Figura 3. Reproducción del fragmento implementado en negocio.py. El archivo completo se entrega con el prototipo.

La función primero valida el cliente y agrupa las cantidades; el fragmento muestra la verificación de existencias, el cálculo y la preparación de la venta. El uso de Decimal desde cadenas evita introducir aproximaciones binarias al representar importes [1]. Se aplica ROUND_HALF_UP al subtotal de cada producto y después se suman los subtotales.

El stock se modifica en nuevo, que es una copia. guardar(nuevo) debe terminar correctamente antes de reemplazar estado. Si falla, la excepción se comunica a la interfaz y el inventario anterior permanece intacto. El código se calcula con el mayor identificador existente, de modo que continúa al volver a abrir el archivo.

Decisiones de legibilidad

Los nombres de las funciones expresan una acción, las reglas comerciales están separadas del menú y los comentarios explican las decisiones menos evidentes. Se evita mezclar la preparación física del producto con una función del programa. Tampoco se confunde una lista en memoria con persistencia: esta última exige escribir y volver a leer un archivo.

11 Archivos y evidencia de ejecución

```
    except (ValueError, KeyError, TypeError, ArithmeticError) as error:
        raise ValueError('Archivo inválido. Se conserva sin cambios; revise una copia de
respaldo.') from error


def guardar(ruta, datos):
    ruta = Path(ruta)
    validar(datos)
    ruta.parent.mkdir(parents=True, exist_ok=True)
    temporal = None
    try:
        with NamedTemporaryFile('w', encoding='utf-8', dir=ruta.parent, delete=False,
suffix='.tmp') as f:
            temporal = f.name
            json.dump(datos, f, ensure_ascii=False, indent=2)
            f.flush()
            os.fsync(f.fileno())
        os.replace(temporal, ruta)
    finally:
        if temporal and os.path.exists(temporal):
            os.unlink(temporal)
```

Figura 4. Guardado implementado en archivos.py con validación y reemplazo mediante un temporal.

Uso del prototipo

Para iniciar, se ejecuta python main.py desde la carpeta del proyecto. La opción 1 solicita cliente, canal y líneas del pedido; el producto 0 termina la captura. Las opciones 2 y 3 buscan y ordenan; la 4 muestra existencias y la 5 calcula el resumen de la fecha local. Cada venta válida se guarda automáticamente en datos/queseria.json.

El archivo de datos incluye información comercial, por lo que se excluye del repositorio mediante .gitignore. Los ejemplos de prueba usan nombres de demostración. No se solicita DNI, dirección ni teléfono. El programa no incorpora permisos por usuario ni cifrado y debe mantenerse en un entorno de prueba controlado.

Control de versiones y trabajo del equipo

Git permite registrar cambios locales y GitHub alojar el repositorio compartido. Para el trabajo grupal se propone separar las tareas de validación, persistencia, interfaz y pruebas; cada integrante debe crear su rama con su identidad real, revisar los cambios y registrar commits descriptivos. El historial se consulta con git log --oneline --graph --all [5].

Las capturas deben mostrar el historial fechado, los cambios de cada aporte y el repositorio remoto. El enlace y las evidencias de participación quedan reservados en el anexo C. Un conjunto de archivos entregado no demuestra por sí mismo trabajo progresivo ni colaboración; ese requisito se acredita con el historial real.

12 Casos de prueba y resultados

Se ejecutaron 18 pruebas automatizadas en Python. Los ejemplos provienen del informe base y se complementan con datos sintéticos para comprobar límites y errores. Cada prueba inicia con un estado independiente; por ello, los saldos de un caso no se trasladan al siguiente. No se presentan estas ejecuciones como pruebas de campo con clientes reales.

Caso	Entrada o condición	Resultado comprobado
01	Daniel; fresco 15 kg a S/ 18.	S/ 270,00 y stock final 5 kg.
02	María; mantecoso 20 kg; stock 15.	Rechazo; estado sin cambios.
03	Yandel; suizo 5 kg y mantequilla 5 kg.	Total S/ 265,00.
04	Nombre en blanco.	Rechazo por nombre inválido.
05	Cantidad de −5 kg.	Rechazo por formato inválido.
06	Luis; suizo 3 kg; stock cero.	Rechazo por falta de stock.
07	Fresco 20 kg; stock 20 kg.	Aceptado; stock final cero.
08	0, letras, NaN, infinito y 1,0001.	Todas las entradas rechazadas.
09	Código existente con espacios y uno ausente.	Encuentra el primero; el segundo no.
10	Tres códigos en orden inverso.	Orden ascendente sin alterar el origen.
11	Guardar venta y leer el archivo de nuevo.	Datos recuperados iguales al estado.
12	Archivo con JSON mal formado.	Rechazo; archivo original conservado.
13	Fallo de escritura simulado.	Sin venta confirmada ni cambio de stock.
14	Fresco 15 kg y 10 kg en el mismo pedido.	Agrupa 25 kg; rechaza por stock.
15	Fresco 0,125 kg.	Aceptado; total S/ 2,25.
16	Resumen de fecha de venta y fecha sin ventas.	S/ 18,00 en la primera; cero pedidos en la otra.
17	Carrito vacío, producto 99 y canal web.	Rechazo en los tres escenarios.
18	Archivo con total manipulado a 999.	Rechazo por inconsistencia del total.

Resultado global: 18 pruebas aprobadas, sin fallas en esta ejecución. La verificación cubre las reglas indicadas; no acredita rendimiento con grandes volúmenes, seguridad para producción ni funcionamiento simultáneo de varias cajas.

13 Evidencias de pruebas y funcionamiento

```
test_01_venta_normal (test_prototipo.Pruebas.test_01_venta_normal) ... ok
test_02_stock_insuficiente (test_prototipo.Pruebas.test_02_stock_insuficiente) ... ok
test_03_compra_multiple (test_prototipo.Pruebas.test_03_compra_multiple) ... ok
test_04_nombre_vacio (test_prototipo.Pruebas.test_04_nombre_vacio) ... ok
test_05_cantidad_negativa (test_prototipo.Pruebas.test_05_cantidad_negativa) ... ok
test_06_stock_cero (test_prototipo.Pruebas.test_06_stock_cero) ... ok
test_07_stock_exacto (test_prototipo.Pruebas.test_07_stock_exacto) ... ok
test_08_numeros_invalidos (test_prototipo.Pruebas.test_08_numeros_invalidos) ... ok
test_09_busqueda (test_prototipo.Pruebas.test_09_busqueda) ... ok
test_10_ordenamiento (test_prototipo.Pruebas.test_10_ordenamiento) ... ok
test_11_persistencia (test_prototipo.Pruebas.test_11_persistencia) ... ok
test_12_archivo_corrupto (test_prototipo.Pruebas.test_12_archivo_corrupto) ... ok
test_13_fallo_guardado (test_prototipo.Pruebas.test_13_fallo_guardado) ... ok
test_14_producto_repetido (test_prototipo.Pruebas.test_14_producto_repetido) ... ok
test_15_coma_decimal (test_prototipo.Pruebas.test_15_coma_decimal) ... ok
test_16_resumen_por_fecha (test_prototipo.Pruebas.test_16_resumen_por_fecha) ... ok
test_17_canal_producto_pedido (test_prototipo.Pruebas.test_17_canal_producto_pedido) ... ok
test_18_archivo_semanticamente_invalido
(test_prototipo.Pruebas.test_18_archivo_semanticamente_invalido) ... ok

----------------------------------------------------------------------
Ran 18 tests in 0.076s

OK
```

Figura 5. Salida real capturada de python -m unittest -v. Los archivos temporales se eliminan al terminar cada prueba.

Comprobación de una venta en consola

```
PED-000001 | Daniel | Total S/ 270.00
Queso fresco: 15 kg x S/ 18 = S/ 270.00
Inventario después de la venta
1 Queso fresco: 5 kg | S/ 18/kg
2 Queso mantecoso: 15 kg | S/ 22/kg
3 Queso suizo: 12 kg | S/ 28/kg
4 Mantequilla: 25 kg | S/ 25/kg
Resumen del día: 1 pedido | S/ 270.00
```

Extracto de la ejecución capturada. La captura completa de salida de consola se conserva en el paquete del prototipo.

Interpretación de los resultados

El caso normal confirma simultáneamente importe y saldo. El caso de producto repetido comprueba un riesgo que no aparece al validar cada línea por separado. La simulación de fallo de escritura muestra que no se confirma una venta que no pudo guardarse. La revisión del total al cargar evita aceptar un archivo con importes internamente inconsistentes.

14 Aprendizaje y reflexión técnica y ética

Estrategias de aprendizaje aplicables al proyecto

El trabajo se organizó a partir de ejemplos pequeños: una venta, una consulta y una escritura de archivo. Esta separación permite comparar cada salida con un resultado esperado antes de integrar el menú. La lectura de documentación oficial se concentra en la necesidad concreta: precisión de importes, representación del estado y pruebas repetibles. Los casos de prueba se conservan para volver a ejecutarlos después de cada cambio.

Necesidad	Estrategia y aplicación	Evidencia
Importes precisos	Consultar Decimal y crear valores desde texto [1].	Caso 15 y cálculo de subtotales.
Conservar ventas	Estudiar JSON y separar lectura y escritura [2].	Caso 11 y módulo archivos.py.
Evitar confirmaciones fallidas	Revisar reemplazo de archivos y trabajar sobre una copia [3].	Caso 13.
Comprobar cambios	Usar unittest con datos aislados [4].	18 pruebas ejecutables.

Estos recursos permiten ampliar el uso básico de funciones, listas y archivos. El equipo deberá contrastarlos con los temas efectivamente vistos en clase antes de declarar cuáles constituyen conocimiento nuevo. La evidencia individual debe describir qué comprendió cada integrante, qué cambió y cómo lo verificó; no basta con adjuntar una captura.

Reflexión técnica

La mejora principal frente a la propuesta inicial es conservar el estado y tratar cada pedido como una operación completa. La simplicidad ayuda a revisar la lógica, pero también impone límites: el archivo crece con las ventas, la búsqueda recorre la lista y no hay coordinación entre varias cajas. La siguiente versión debería incorporar reposición y anulaciones trazables antes de utilizarse para controlar un negocio real.

Reflexión ética y social

Un saldo incorrecto puede hacer que el personal prometa un producto que no tiene, mientras que un importe mal calculado afecta al cliente o al negocio. Por eso se rechaza el pedido completo ante una inconsistencia y se informa el error. El ahorro de tiempo es un beneficio esperado, pero no se afirma como medido hasta contar con observaciones comparables.

Los nombres y las compras no deben publicarse en el repositorio. Las fotografías de campo deben tomarse con autorización y ocultar datos de clientes. Asimismo, la elaboración asistida por IA debe declararse: el grupo conserva la responsabilidad de revisar el programa y explicar sus decisiones durante la sustentación.

15 Conclusiones y recomendaciones

Conclusiones

El prototipo cubre el registro, la validación, la búsqueda, el ordenamiento y el almacenamiento de pedidos para el caso comercial autorizado. Los cálculos y las restricciones de stock se expresan mediante reglas comprobables. Las listas, funciones, cadenas, estructuras de control y archivos tienen una función concreta en la solución.

Las 18 pruebas ejecutadas comprobaron escenarios normales, límites y errores, incluidos el guardado y la recuperación del estado. El resultado respalda el comportamiento de esta versión con los casos documentados. No permite concluir que la aplicación esté lista para varias cajas ni que haya reducido tiempos reales de atención.

La información de campo y el historial de colaboración complementan la evidencia técnica. El prototipo, las pruebas y los diagramas permiten revisar la solución; las fotografías fechadas, entrevistas y aportes en GitHub deben acreditar el proceso real del grupo.

Recomendaciones técnicas y conocimiento complementario

Se recomienda conservar Decimal para los importes [1] y documentar la regla de redondeo. Su aplicación evita mezclar montos monetarios con cálculos aproximados innecesarios. La prueba con 0,125 kg muestra un caso decimal concreto; conviene añadir casos de redondeo a medio céntimo al modificar precios.

El guardado mediante temporal y reemplazo [3] amplía la escritura directa de archivos: primero prepara el estado completo y después sustituye el destino. Debe complementarse con respaldos y una prueba de recuperación, porque un reemplazo correcto no protege frente a borrado accidental o falla del equipo.

Las pruebas con unittest [4] deben ejecutarse después de cada cambio. Antes de incorporar reposiciones o anulaciones, se deben definir sus reglas y verificar que stock, historial y total permanezcan consistentes. Para varios usuarios será necesario revisar el modelo de almacenamiento y la concurrencia, no solo cambiar la apariencia de la interfaz.

Verificación de campo y preparación de la sustentación

Se recomienda medir varias atenciones con un mismo criterio de inicio y fin, registrar el número de observaciones y comparar periodos semejantes. En la exposición, el equipo debe demostrar una venta válida, un rechazo por stock, una búsqueda y la recuperación de datos tras reiniciar. También debe explicar por qué la venta se guarda antes de actualizar la memoria.

Para la entrega final se adjuntan el informe editable, su versión PDF, una presentación de hasta 12 diapositivas y el código con pruebas. El grupo completará los campos reservados de autoría, campo y repositorio, y adjuntará la declaración oficial de IA disponible en el aula virtual.

16 Referencias

Documentos del curso y antecedente

Universidad Privada del Norte. (s. f.). Evaluación final del curso Fundamentos de Programación CIIN1205P. Desafío Del archivador al algoritmo, un prototipo para la mesa de partes digital. Documento proporcionado para la actividad.

Grupo 1. (2026). Informe de proyecto final sobre la Quesería Los Alpes. Documento base proporcionado para la revisión. Sus ejemplos de precios y existencias se utilizan como datos de demostración.

Fuentes técnicas consultadas

[1] Python Software Foundation. (s. f.). decimal — Decimal fixed-point and floating-point arithmetic. https://docs.python.org/3/library/decimal.html

[2] Python Software Foundation. (s. f.). json — JSON encoder and decoder. https://docs.python.org/3/library/json.html

[3] Python Software Foundation. (s. f.). os — Miscellaneous operating system interfaces. Apartado os.replace. https://docs.python.org/3/library/os.html#os.replace

[4] Python Software Foundation. (s. f.). unittest — Unit testing framework. https://docs.python.org/3/library/unittest.html

[5] Git. (s. f.). git-log Documentation. https://git-scm.com/docs/git-log

Fecha de consulta: 14 de octubre de 2026. El prototipo se verificó con el intérprete de Python disponible en el entorno de trabajo y utiliza únicamente su biblioteca estándar.

Correspondencia con el desafío autorizado

Se aplican al proceso comercial las exigencias de análisis, alternativas, diseño, programación, pruebas y aprendizaje. La referencia municipal y sus obligaciones normativas pertenecen al caso original del desafío y no se trasladan a la quesería. La adaptación del caso responde a la autorización del docente comunicada por el grupo.

Anexo A Instrumentos para la práctica de campo

Ficha de observación y bitácora fotográfica. Completar durante la visita con información verificable.

Institución y dirección: _____

Fecha y horario: _____

Responsable que autoriza la visita: _____

Integrantes presentes: _____

Registro	Qué documentar	Dato o evidencia
F01	Recepción del pedido y formato utilizado.	Fecha y archivo: _____
F02	Consulta de existencias, sin datos personales.	Fecha y archivo: _____
F03	Registro y cálculo del importe.	Fecha y archivo: _____
F04	Actualización y revisión al cierre.	Fecha y archivo: _____

Para cada fotografía: registrar fecha, lugar, actividad, autorización y descripción. Ocultar nombres, teléfonos, comprobantes y cualquier dato identificable de clientes.

Atención	Inicio	Fin	Duración	Incidencia
1	_____	_____	_____	_____
2	_____	_____	_____	_____
3	_____	_____	_____	_____

Criterio de medición: inicio cuando el cliente comunica su pedido; fin cuando se confirma su aceptación o rechazo. Registrar aparte preparación y entrega. Duración = hora final − hora inicial. Ampliar la ficha si se observan más atenciones.

Guía breve de entrevista

1. ¿Qué datos se anotan y cuáles suelen faltar?
2. ¿Cómo se confirma el stock y quién lo actualiza?
3. ¿Qué ocurre cuando un pedido incluye varios productos?
4. ¿Qué errores se repiten al cobrar o cerrar la jornada?
5. ¿Cómo se consultan pedidos anteriores y cómo se conservan?
6. ¿Qué cambio sería más útil y quién validaría el prototipo?

Nombre o cargo y consentimiento: _____
Respuestas y notas: _____
_____

Anexo B Declaración de uso de inteligencia artificial

Texto de apoyo para trasladar al formato oficial del aula virtual. Esta hoja no reemplaza el formato institucional obligatorio ni una firma de los integrantes.

Herramienta y finalidad

Se utilizó ChatGPT en el entorno Codex para revisar los documentos proporcionados, identificar los requisitos del desafío, reorganizar la redacción y elaborar el código del prototipo, diagramas, pruebas y materiales de sustentación. El apoyo comprendió generación de contenido y programación, además de corrección de estilo.

Verificaciones realizadas durante la preparación

Se contrastó el contenido con el desafío y el informe base, se verificaron los cálculos de los ejemplos y se ejecutaron 18 pruebas automatizadas sobre el programa. Los datos de ventas utilizados son ejemplos de demostración; no se generaron fotografías de campo, entrevistas ni aportes ficticios de integrantes.

Revisión y responsabilidad del grupo

Antes de firmar la declaración, cada integrante debe revisar el contenido, ejecutar o presenciar la demostración y comprender las funciones que presenta. Las correcciones propias y las verificaciones adicionales se registrarán en los espacios siguientes.

Integrantes que revisaron: _____
Fecha de revisión: _____
Cambios realizados por el equipo: _____
_____
Comprobaciones adicionales: _____
_____
Firmas según formato oficial: _____

Registro individual de aprendizaje

Integrante	Conocimiento nuevo	Aplicación y evidencia
_____	_____	_____
_____	_____	_____
_____	_____	_____
_____	_____	_____
_____	_____	_____

Anexo C Control de entrega y evidencias del grupo

Requisito	Ubicación o estado
Caso autorizado y justificación	Secciones 1 y 2.
Entradas, procesos y salidas	Sección 3.
Dos alternativas y contraste con tercerizar	Sección 4.
Principios matemáticos y de ingeniería	Sección 5.
Diagrama de flujo y pseudocódigo	Secciones 6 a 9.
Funciones, arreglos, cadenas y archivos	Secciones 5 y 10 a 11; código adjunto.
Resultados de pruebas ejecutadas	Secciones 12 y 13; 18 pruebas.
Reflexión técnica y ética	Sección 14.
Aprendizaje y conocimiento complementario	Secciones 14 y 15; registro individual en anexo B.
Observación, fotos fechadas y entrevista	Completar con evidencias reales en anexo A.
Pruebas con datos recogidos en campo	Añadir al validar la información de la visita.
Git, GitHub e historial progresivo	Completar enlace y capturas reales debajo.
Informe PDF y presentación	Archivos adjuntos con el nombre del grupo.
Declaración oficial de IA	Trasladar anexo B al formato del aula y adjuntarlo.
Integrantes y participación	Completar carátula con datos reales.

Enlace al repositorio: _____
Rama o versión entregada: _____
Identificador del commit final: _____

Adjuntar capturas legibles del historial fechado, aportes por integrante y vista del repositorio remoto. No publicar archivos con datos reales de clientes.

Entrega: aula virtual, según el sílabo, semanas 15 y 16. Límite del desafío: 20 páginas de contenido sin contar carátula ni anexos y 12 diapositivas. Revisar que los archivos correspondan a la misma versión antes de subirlos.